

Fast Confirmation Rule for Ethereum’s Consensus Protocol

A Short Explainer

Mikhail Kalinin
Consensys

Roberto Saltini
Ethereum Foundation

Luca Zanolini
Ethereum Foundation

The confirmation rule described in this document is based on the reference implementation available at: <https://github.com/ethereum/consensus-specs/pull/4747>.

1 Strictly-Essential Notation

Ethereum’s proof-of-stake protocol *Gasper* combines LMD-GHOST-HFC for fork-choice and FFG-Casper for finality. We introduce only the notation and assumptions needed for the Confirmation Rule.

Time. Gasper advances in discrete *slots*; every E slots form an *epoch*.

- $\text{slot}(t)$: slot active at time t .
- $\text{epoch}(s)$: epoch of slot s .
- E : slots per epoch.
- $\text{st}(\text{slot}(\cdot))$: beginning of a slot.
- $\text{fslot}(\text{epoch}(\cdot))$: first slot of an epoch.
- $\text{lslot}(\text{epoch}(\cdot))$: last slot of an epoch.

Blocks and ancestry. Blocks form a tree via parent links; honest validators extend the heaviest canonical chain.

- $\text{p}(b)$: parent of block b .
- $b_a \prec b_d$: b_a is an ancestor of b_d .
- $\text{chain}(b)$: ancestors of b (incl. b_{gen}).
- $\text{epoch}(b)$: epoch of b .

Checkpoints. Checkpoints summarize the chain once per epoch and are the objects justified and finalized by FFG-Casper.

- $\text{C}(b, e)$: checkpoint of b at epoch e .
- $\text{C}(b)$ is defined as $\text{C}(b, \text{epoch}(b))$: latest checkpoint of b .
- $\text{block}(C)$: block of checkpoint C
- $\text{epoch}(C)$: epoch of checkpoint C
- $b \preceq C$ is defined as $b \preceq \text{block}(C)$
- $C \prec b$ is defined as $\text{C}(b, \text{epoch}(C)) = C$
- $C_1 \prec C_2$ is defined as $\text{epoch}(C_1) < \text{epoch}(C_2)$ and $C_1 \prec \text{block}(C_2)$: checkpoint order in the chain.

Votes, views, and fork-choice. LMD-GHOST-HFC selects the canonical head from weighted block votes, while FFG-Casper finalizes checkpoints via supermajority links.

- GHOST vote: block-level vote.
- FFG vote: checkpoint-level vote.
- $\mathcal{V}$: a validator’s current message view.
- $\text{FC}_{\mathfrak{B}}^v(t)$: head in v ’s view at time t .
- $F_{\rightarrow C}^{s,v,t,C}$: total weight of all FFG votes with any source and target C sent at time t .

Checkpoints hierarchy.

- $\text{GJ}(b)$: greatest realized justified checkpoint in $\text{chain}(b)$.
- $\text{GU}(b)$: greatest *unrealized justified* checkpoint (not yet justified in $\text{chain}(b)$, but it will be on epoch crossing).
- $\text{AU}(b)$: set of *unrealized justified* checkpoints in $\text{chain}(b)$ (as determined by the FFG votes included in blocks).

Voting source and local justification.

- $\text{vs}(b, t)$: source checkpoint for b ’s FFG vote, at time/epoch of t^1 .
- $\text{GJ}^{t,v}$: validator v ’s greatest justified checkpoint at time t .

Adversarial bound.

- β : adversarial weight fraction ($< \frac{1}{3}$).
- GST : time after which all honest views align.

2 Assumptions

The following assumptions are required for our Algorithm. We state them informally here, and they are used throughout the document.

Assumption 1 (Static validator set). *Validator set and balances are static except for slashed Byzantine validators. No new joins or exits. This simplifies analysis without changing behavior.*

Assumption 2 (Adversarial fraction). *For $t \geq \text{GST}$ and any checkpoint C justified in the chain of a valid block, the fraction of adversarial validators in any set of consecutive committees is at most β .*

This mirrors Ethereum’s $\beta' < 1/3$ safety bound; with $\sim 10^6$ validators and $E = 32$, committee sampling ensures $\beta < 1/3$ with overwhelming probability.

Assumption 3 (FFG behavior).

1. $\beta < \frac{1}{3}$.
2. *Byzantine validators cannot block honest FFG votes from inclusion in canonical blocks for an entire epoch.*

In practice, the latter may fail only due to vote-capacity limits per block; without that restriction it suffices to assume one honest proposer per epoch.

¹https://github.com/ethereum/consensus-specs/blob/master/specs/phase0/fork-choice.md#get_voting_source

3 Confirmation Rule

Goal. We need a simple, local condition telling validator v : “If I confirm block b now, then under standard assumptions b will remain the LMD-GHOST head for all honest validators from the next slot onward.” We introduce a *confirmation rule*.

A confirmation rule is an algorithm on top of the fork-choice deciding when a block is *confirmed*, i.e., guaranteed to stay in canonical chain of every honest validator under standard assumption.

A valid confirmation rule must satisfy two properties: *safety*, meaning that if an honest validator confirms b at time t , then every honest validator will keep b in its canonical chain at all future times; and *monotonicity*, meaning that once a block becomes confirmed, it remains confirmed for all $t' \geq t$ and is never revoked.

3.1 Confirmation Rule for LMD-GHOST

Why start with plain LMD-GHOST. The final goal is a confirmation rule for LMD-GHOST-HFC, the fork-choice that filters out blocks conflicting with justified checkpoints and is the version currently implemented in Ethereum’s consensus protocol. Reasoning directly about this filter is cumbersome because it interacts with both fork-choice votes and FFG finality. We therefore begin with the unfiltered LMD-GHOST function, deriving a local, vote-weight condition that guarantees stable heads. In the next section, we extend this rule so confirmed blocks also survive LMD-GHOST-HFC’s checkpoint filtering.

Locally measurable inputs at time t . All quantities are computed by v using votes up to slot $\text{slot}(t)-1$ and the balance assignment anchored at checkpoint C^2 .

- $S_b^{s,v,t,C}$: total weight of GHOST votes (as seen by v) supporting descendants of a given ancestor over slots $[\text{slot}(b), s]$.
- $W_b^{s,C}$ denotes the total committee weight over all slots in the interval $[\text{slot}(b), s]$, computed using the balance assignment defined by checkpoint C . We also use the notation $W_{s_1}^{s_2,C}$, where the range is specified directly by slots rather than by blocks. The meaning is identical, and the distinction between the two forms will always be clear from context.
- $W_{b+}^{s,C}$: total committee weight over slots $[\text{slot}(b) + 1, s]$, under the balance assignment from C .
- W_p^C : proposer boost (a fixed fraction of average committee weight for the epoch derived from C).
- $I_b^{s,v,t,C}$: total weight of validators that have committed a slashable offense.
- β : adversarial weight bound ($< \frac{1}{3}$).

The predicate. For block b and checkpoint C , define:

$$\begin{aligned} \text{isOneConfirmed}_v(b, C, t) := Q_b^{\text{slot}(t)-1,v,t,C} &> \frac{1}{2} \left(1 + \frac{W_p^C}{W_{p(b)+}^{\text{slot}(t)-1,C}} \right) \\ &+ \beta - \frac{I_b^{\text{slot}(t)-1,v,t,C}}{W_{p(b)+}^{\text{slot}(t)-1,C}} \end{aligned}$$

where

$$Q_b^{s,v,t,C} := \frac{S_b^{s,v,t,C}}{W_{p(b)+}^{s,C}}.$$

²All vote weights are based on validator balances from the latest justified checkpoint C , which fixes the active set and balances. In practice, C is the greatest *unrealized justified* checkpoint—already justified but not yet finalized—ensuring consistent weighting across validators.

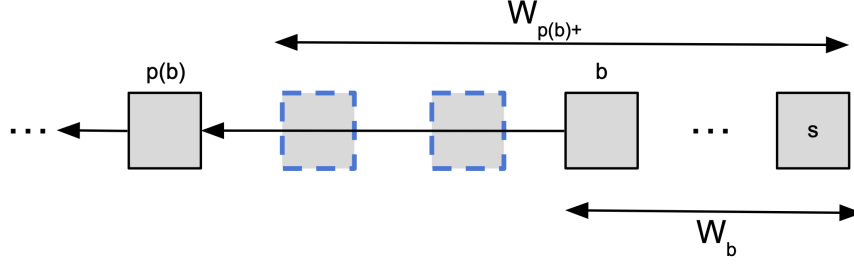


Figure 1: Quantities $W_b^{s,C}$ and $W_{b'+}^{s,C}$ with respect to the current slot s .

and

$$\text{isLMDGHOSTSafe}^v(b, C, t) := \forall b' \preceq b, \text{isOneConfirmed}_v(b', C) \\ \text{or } b' = b_{\text{gen}}$$

Intuition and interpretation. This predicate asks:

“At time t , using balance weights from checkpoint C , does the voting evidence show that every ancestor of b already commands a strict majority of total committee weight, by a margin wide enough to absorb both the proposer boost and all possible Byzantine votes?”

Equivalently: “Is my current head b so dominantly supported that, no matter what new honest or adversarial votes appear later, the LMD-GHOST fork-choice will keep following this same path?”

The ratio

$$\frac{S_b^{s,v,t,C}}{W_{p(b)+}^{s,C}}.$$

measures the fraction of total voting weight currently favoring the subtree containing b . Checking the condition for all $b' \preceq b$ guarantees that every decision point in the recursive LMD-GHOST walk is stable, so future votes cannot redirect the fork-choice to a competing branch.

Each term in the inequality has a specific role:

- $\frac{1}{2}$: basic majority threshold for subtree dominance;
- $\frac{W_p}{\text{committee weight}}$: compensates for proposer boost in block scoring;
- $+\beta$: margin of safety covering potential Byzantine voters;
- $-\frac{I}{\text{committee weight}}$: this term represents the fraction of slashed validators. Since slashed validators are provably Byzantine, their weight can be subtracted from the adversarial allowance β .

If the inequality holds for all ancestors, b is *LMD-safe* – it will remain on the canonical chain for all honest validators.

Empty slots. The `isLMDGHOSTSafe` predicate can also be strengthened to account for *empty slots*, i.e., slots in which no block was proposed. The resulting expression is more involved, so instead of presenting the full formula we illustrate the intuition through a concrete example below.

Consider a block b at slot 10 whose parent lies at slot 7, so slots 8 and 9 contain no block on the chain of b .

Let $S(b')_8^{9,v,t,C}$ denote the validators who, in the view of v , cast a GHOST vote in slots 8 and 9 for b ’s parent b' . From this set, we subtract a worst-case Byzantine contribution $\beta W_8^{9,C}$, obtaining a lower bound on the *honest* validators who voted for the parent of b in the missing slot.

This lower bound,

$$H(b')_8^{9,v,t,C} = \max\left(S(b')_8^{9,v,t,C} - \beta W_8^{9,C} + I_8^{9,v,t,C}, 0\right),$$

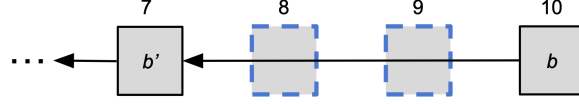


Figure 2: Missing slot case: b at slot 10 whose parent lies at slot 7, so slots 8 and 9 contain no block on the chain of b .

captures the *minimum guaranteed honest support* in slots 8 and 9 for the branch rooted at b' . Here:

- $S(b')_8^{9,v,t,C}$ counts validators whose slot-8 and slot-9 votes stay on the parent branch,
- $\beta W_8^{9,C}$ subtracts the maximum weight that could be adversarial,
- $I_8^{9,v,t,C}$ adds back the weight of slashable equivocators, who can no longer contribute adversarially.

These honest voters are guaranteed *not* to appear later as supporters of a conflicting branch, since they already voted in favor of a prefix of b .

Because they cannot contribute to any conflicting subtree, the safety threshold can safely *discount* this weight from the proposer–boost term in the numerator of the isLMDGHOSTSafe inequality.

Intuitively, whenever a block has empty intermediate slots, the improved formula accounts for honest votes cast in the empty slots for the parent of b , preventing the protocol from pessimistically assuming that all votes in those slots may still swing to a conflicting branch.

In the modified isLMDGHOSTSafe predicate, the term involving β is further multiplied by

$$\frac{W_{b'}^{\text{slot}(t)-1,v,t,C}}{W_{p(b')+}^{\text{slot}(t)-1,C}},$$

a fraction that is always ≤ 1 . In this fraction, the numerator represents the total committee weight from the slot of block b' up to slot b , while the denominator represents the total committee weight over all slots after the parent of b' .

This scaling reflects that, although β bounds the adversarial proportion *among all voters*, only a subset of them could have cast votes relevant for block b .

Crucially, validators in slot 9 *cannot have voted for b* , since b is a slot-10 block. Yet the denominator of Q counts votes from slot 9. Therefore, the adversarial budget for slot 9 can safely be reduced: the only validators who could harm b are those who voted for a conflicting block *at or after* the slot of b , not those who voted strictly before it. The multiplicative factor above precisely downscales the β term to only the portion of validators whose votes could actually conflict with b .

Complete predicate. We now present the complete formula for the isLMDGHOSTSafe predicate. Although it relies on several auxiliary definitions, it precisely captures the general case including missing-slot handling:

$$\begin{aligned} \text{isOneConfirmed}_v(b, C, t) := Q_b^{\text{slot}(t)-1,v,t,C} &> \frac{1}{2} \left(1 + \frac{W_p^C - H(p(b))_{\text{slot}(p(b))+}^{\text{slot}(b)-1,v,t,C}}{W_{p(b)+}^{\text{slot}(t)-1,v,t,C}} \right) \\ &+ \beta \frac{W_b^{\text{slot}(t)-1,v,t,C}}{W_{p(b)+}^{\text{slot}(t)-1,C}} - \frac{I_b^{\text{slot}(t)-1,v,t,C}}{W_{p(b)+}^{\text{slot}(t)-1,C}} \end{aligned}$$

The term

$$H(p(b))_{\text{slot}(p(b))+}^{\text{slot}(b)-1,v,t,C} = \max\left(S(p(b))_{\text{slot}(p(b))+}^{\text{slot}(b)-1,v,t,C} - \beta W_{p(b)+}^{\text{slot}(b)-1,C} + I_{p(b)+}^{\text{slot}(b)-1,v,t,C}, 0\right)$$

corresponds exactly to the quantity denoted by H in our example: a *lower bound on the weight of honest validators* whose votes in the empty slots are for b (after correcting for the worst-case β fraction of Byzantine voters and restoring the weight of slashed equivocators).

The set

$$S(\mathbf{p}(b))_{\text{slot}(\mathbf{p}(b))+}^{s,v,t,C}$$

used inside this expression plays the role of the set S in the earlier example: it contains all validators whose GHOST vote is for $\mathbf{p}(b)$. These less the possible Byzantine ones are precisely the validators whose weight must be conservatively discounted when establishing safety for block b .

Guarantees. The following guarantees hold as long as the checkpoint used to evaluate *isOneConfirmed* is an ancestor of the checkpoint used by honest validators to evaluate the fork-choice function:

- **Head safety:** $\text{isLMDGHOSTSafe}(b, C, t) = \text{True} \Rightarrow b$ is the LMD-GHOST head for v at t .
- **Future agreement:** From the next slot, all honest validators will also select b .
- **Monotonicity:** As more honest votes accumulate, Q can only increase; once b becomes safe, it remains safe³.

3.2 Confirmation Rule for LMD-GHOST-HFC

When moving to LMD-GHOST-HFC, the predicate *isLMDGHOSTSafe* alone is no longer sufficient. Even if satisfied, block b may later be filtered out, e.g., if it is not a descendant of the greatest justified checkpoint and either conflicts with it, or has no descendant whose voting source⁴ is that checkpoint or has epoch $\geq \text{epoch}(\text{now}) - 2$.

tl;dr Algorithm 1 – the pseudocode is based on the Python spec <https://github.com/ethereum/consensus-specs/pull/4747> – first checks whether the current confirmation state is still valid. If the candidate block b_c is too old, no longer canonical, or fails the local safety check at the epoch boundary, confirmation resets to the *greatest finalized block*, recovering safely from asynchrony or temporary forks.

Confirmation *can start or restart only at the beginning of an epoch*. At that moment, if the greatest unrealized justified checkpoint from the previous epoch was already observed at the beginning of the last slot of that epoch, then, upon entering the new epoch, that checkpoint has now become realized as justified for all honest validators.

The algorithm anchors on the corresponding block and begins the confirmation process from there – a guaranteed safe starting point that will not be filtered out by LMD-GHOST-HFC.

From this anchor, it calls *find_latest_confirmed_descendant*, which walks forward along the canonical chain, checking each child block with the single-block *isOneConfirmed* predicate. If the condition holds, the block becomes confirmed; if it fails, the scan stops. This produces a monotone, slot-by-slot confirmation that follows the canonical chain and that checks that the block cannot be filtered out by the FFG filtering rules.

As time advances, the process may also *extend across epochs*, but only if no conflicting justification is expected.

Overall, the procedure combines:

- epoch-boundary anchoring on the newly realized justified checkpoint,
- incremental local confirmation via *isOneConfirmed*,
- and strict cross-epoch guards against conflicting justification.

The output is the deepest block guaranteed to remain canonical under both LMD-GHOST and FFG.

³This guarantee holds only within a single epoch; ensuring it across epochs requires additional mechanisms.

⁴The source checkpoint according to the view in block b .

Algorithm 1: Confirmation Rule

```

1: State:
2:    $b_{\text{confirmed}}^v$ 
3: upon  $\text{now}^v = t_{\text{init}}^v$ 
4:    $b_{\text{confirmed}}^v \leftarrow \text{block}(\text{GF}^{\text{now}^v, v})$ 
5: end
6: upon  $\text{now}^v = \text{st}(\text{slot}(\text{now}^v)) \wedge \text{now}^v \geq t_{\text{init}}^v$ 
7:    $b_{\text{confirmed}}^v \leftarrow \text{get\_latest\_confirmed}^v(b_{\text{confirmed}}^v)$ 
8: end
9: function  $\text{is\_chain\_one\_confirmed}^v(b_c, C)$ 
10:    $b \leftarrow b_c$ 
11:   while  $b \neq \text{block}(C) \wedge \text{isOneConfirmed}^v(b, C)$ 
12:      $b \leftarrow \text{p}(b)$ 
13:   return  $b = \text{block}(C)$ 
14: end
15: function  $\text{willNoConflictingChkpBeJustified}^v(C)$ 
16:   return  $\max(F_{\rightarrow C}^{\text{slot}(\text{now}^v)-1, v, \text{now}^v, C} - \beta W_{\text{fslot}(\text{epoch}(t)), 0}^{\text{slot}(t)-1, C}, 0) + (1 - \beta) W_{\text{slot}(\text{now}^v)}^{\text{fslot}(\text{epoch}(C)), C} \geq \frac{1}{3} W_t^C$ 
17: end
18: procedure  $\text{get\_latest\_confirmed}^v(b_c)$ 
19:    $b_{\text{cand}} \leftarrow b_c$ 
20:   const  $\text{head} \leftarrow \text{LMD-GHOST-HFC}(\mathcal{V}^{v, \text{now}^v})$ 
21:   if  $\text{epoch}(b_{\text{cand}}) < \text{epoch}(\text{now}^v) - 1$  or  $b_{\text{cand}} \not\preceq \text{head}$  or  $(\text{slot}(\text{now}^v) = \text{fslot}(\text{epoch}(\text{now}^v)) \text{ and } b_{\text{cand}} \succeq \text{GU}^{\text{st}(\text{fslot}(\text{epoch}(t))-1), v} \text{ and } \neg \text{is\_chain\_one\_confirmed}^v(b_{\text{cand}}, \text{GU}^{\text{st}(\text{fslot}(\text{epoch}(t))-1), v}))$  then
22:      $b_{\text{cand}} \leftarrow \text{block}(\text{GF}^{t^v, v})$ 
23:   if  $\text{slot}(t^v) = \text{fslot}(\text{epoch}(t^v))$  and  $\text{epoch}(\text{GU}^{\text{st}(\text{fslot}(\text{epoch}(t))-1), v}) = \text{epoch}(t^v) - 1$  and  $\text{GU}(\text{LMD-GHOST-HFC}(\mathcal{V}^{v, t^v})) = \text{GU}^{\text{st}(\text{fslot}(\text{epoch}(t))-1), v}$  and  $\text{slot}(b_{\text{cand}}) < \text{slot}(\text{block}(\text{GU}^{\text{st}(\text{fslot}(\text{epoch}(t))-1), v}))$  then
24:      $b_{\text{cand}} \leftarrow \text{block}(\text{GU}^{\text{st}(\text{fslot}(\text{epoch}(t))-1), v})$ 
25:   if  $\text{epoch}(b_{\text{cand}}) \geq \text{epoch}(\text{now}^v) - 1$  then
26:      $b_{\text{cand}} \leftarrow \text{find\_latest\_confirmed\_descendant}^v(b_{\text{cand}})$ 
27:   return  $b_{\text{cand}}$ 
28: end

```

Entry point and “active” state. We focus on $\text{get_latest_confirmed}_v(b_c)$, the entry point of the Confirmation Rule, which is run at the beginning of every slot. The confirmation rule enters its *active* state – as opposed to the *inactive* state, where no new blocks are confirmed (as discussed later) – exactly at the beginning of an epoch, provided the following conditions hold:

$$\begin{aligned}
& \text{slot}(\text{now}^v) = \text{fslot}(\text{epoch}(\text{now}^v)) \wedge \\
& \text{epoch}(\text{GU}^{\text{st}(\text{slot}(\text{now}^v)-1), v}) = \text{epoch}(\text{now}^v) - 1 \wedge \\
& \text{GU}(\text{LMD-GHOST-HFC}(\mathcal{V}^{v, \text{now}^v})) = \text{GU}^{\text{st}(\text{slot}(\text{now}^v)-1), v} \wedge \\
& \text{slot}(b_{\text{cand}}) < \text{slot}(\text{block}(\text{GU}^{\text{st}(\text{slot}(\text{now}^v)-1), v})).
\end{aligned}$$

Intuitively: At the *first slot* of epoch e , this condition checks that:

- the current epoch has *exactly* just started;
- the epoch of the greatest unrealized justified checkpoint observed at the last slot of epoch $e-1$ is from epoch $e-1$; and
- the current candidate b_c precedes the newly realized justified checkpoint, i.e., its slot is earlier than the block that now serves as the epoch’s justified anchor: we do not want to go backwards with the

confirmation.

Since justification only becomes effective at epoch boundaries, crossing into epoch e implies that this checkpoint is now *realized as justified* in every honest view and no greater justification can be seen. Because LMD-GHOST-HFC starts its fork-choice traversal from the greatest justified checkpoint, all honest validators will now vote for its descendants, and no block on that branch can be filtered out during epoch e .

When the condition holds, we *(re)start*⁵ the confirmation chain at that checkpoint:

$$b_{\text{cand}} \leftarrow \text{block}(\text{GU}(\mathcal{V}^v, \text{st}(\text{slot}(\text{now}^v) - 1))),$$

and immediately continue by confirming as far as possible along the canonical chain via

$$b_{\text{cand}} \leftarrow \text{find_latest_confirmed_descendant}_v(b_{\text{cand}}).$$

Per-block check. Inside *find_latest_confirmed_descendant*, we advance *one block at a time* using *isOneConfirmed* _{v} (b, C, t) defined above with C being the greatest unrealized justified checkpoint at we saw at the beginning of the last slot of the previous epoch. One of the reasons for this choice is that we need to ensure that the checkpoint that we use to evaluate *isOneConfirmed* is an ancestor of the checkpoint that any honest validator will use to evaluate their fork-choice. The other reason will be explained further down when we describe the failure cases. Note that in the pseudocode we use just *isOneConfirmed* _{v} (b, C), without the t parameter, as there we use the notation now^v to indicate the current time of validator v .

Operationally: if the inequality holds for block b , we tentatively move to the child and recheck for the child; if it ever fails, we stop and return the last block that passed. This yields a monotone, slot-by-slot confirmation that follows the canonical chain while guaranteeing that confirmed blocks remain both LMD-GHOST-stable and unfiltered by LMD-GHOST-HFC.

Finding the latest confirmed descendant. Function *find_latest_confirmed_descendant* advances confirmation forward along the canonical chain, returning the deepest descendant of the last confirmed block b_c that is safe under both LMD-GHOST and the FFG filtering.

The function begins with defining the following two local constants:

- *head* : the current head,
- *phead* : the *previous head*, that is, the head we saw at the beginning of the previous slot.

The function *find_latest_confirmed_descendant* can be viewed as consisting of two major components, namely the two outer if conditions. From a high-level perspective – and assuming that the block we wish to confirm lies in the previous epoch, otherwise we simply execute only the second if – the first if condition is the more conservative one: it allows confirmation to progress even under several mild failure scenarios. The second if condition instead behaves as an “all-or-nothing” gate: it permits a more aggressive fast confirmation when all of its conditions hold, but it does not allow any progress along the canonical chain if they fail.

We examine these various conditions base on whether the current confirmed block is from the previous or current epoch.

Previous Epoch - First if The condition of the first if ensure that, if it holds, then any block along the canonical chain up to the previous head, as long as it belongs to the previous epoch, is not going to be filtered out by the FFG filtering conditions.

This condition considers two cases depending on whether the current time is the beginning of *first slot* of the current epoch.

Case 1: Beginning of the epoch. If $\text{slot}(\text{now}^v) = \text{fslot}(\text{epoch}(\text{now}^v))$ (Figure 3), the rule checks:

$$\text{vs}(\text{phead}, \text{now}^v) \geq \text{epoch}(\text{now}^v) - 2, .$$

This condition ensures that the voting source of *phead* is not older than two epochs, which ensures that the previous head will not be filtered out in the current epoch. This also implies that whatever block we confirm will not be filtered out as we only consider blocks that are ancestor of the previous head.

⁵By restart, we mean resuming confirmation of new blocks after the confirmation rule became inactive for unexpected reasons, such as the onset of an asynchronous period.

Algorithm 2: Find latest confirmed descendant

```

29: function  $next\_child^v(b, head)$ 
30:    $extension = \{b' \in \mathcal{V}^{v, now^v}, b \prec b' \preceq head\}$ 
31:   if  $|extension| > 0$  then
32:     return  $\arg \min_{b' \in extension} slot(b')$ 
33:   else
34:     return  $\perp$ 
35: end
36: function  $find\_latest\_confirmed\_descendant^v(b_c)$ 
37:    $b_{cand} \leftarrow b_c$ 
38:   const  $head \leftarrow \text{LMD-GHOST-HFC}(\mathcal{V}^{v, now^v})$ 
39:   const  $phead \leftarrow \text{LMD-GHOST-HFC}(\mathcal{V}^{v, st(slot(now^v)-1)})$ 
40:   if  $\text{epoch}(b_{cand}) = \text{epoch}(now^v) - 1$  and  $vs(phead, now^v) \geq \text{epoch}(now^v) - 2$  and
     $(slot(now^v) = fslot(\text{epoch}(now^v)) \text{ or } (willNoConflictingChkpBeJustified^v(C(head, \text{epoch}(now^v))))$ 
    and  $(GU(phead) \geq \text{epoch}(now^v) - 1 \text{ or } GU(head) \geq \text{epoch}(now^v) - 1))$  then
41:     while  $next\_child^v(b_{cand}, head) \neq \perp$ 
42:        $b_{temp} \leftarrow next\_child^v(b_{cand}, head)$ 
43:       if  $\text{epoch}(b_{temp}) < \text{epoch}(now^v)$  and  $b_{temp} \preceq phead$  and
         $isOneConfirmed_v(b_{temp}, GU^{fslot(\text{epoch}(t))-1, v})$  then
44:          $b_{cand} \leftarrow b_{temp}$ 
45:       else
46:         break
47:   if  $slot(now^v) = fslot(\text{epoch}(now^v))$  or  $GU(head) \geq \text{epoch}(now^v) - 1$  then
48:      $b_{tentative\_cand} \leftarrow b_{cand}$ 
49:     while  $next\_child^v(b_{tentative\_cand}, head) \neq \perp$ 
50:        $b_{temp} \leftarrow next\_child^v(b_{tentative\_cand}, head)$ 
51:       if  $\text{epoch}(b_{temp}) > \text{epoch}(b_{tentative\_cand})$  and  $\neg willChkpBeJustified_v(C(b_{temp}))$  then
52:         break
53:       if  $isOneConfirmed_v(b_{temp}, GU^{fslot(\text{epoch}(t))-1, v})$  then
54:          $b_{tentative\_cand} \leftarrow b_{temp}$ 
55:       else
56:         break
57:   if  $\text{epoch}(b_{tentative\_cand}) = \text{epoch}(now^v)$  or  $(vs(b_{tentative\_cand}, now^v) \geq \text{epoch}(now^v) - 2 \text{ and}$ 
     $(slot(now^v) = fslot(\text{epoch}(now^v)) \text{ or}$ 
     $willNoConflictingChkpBeJustified_v(C(head, \text{epoch}(now^v))))$  then
58:      $b_{cand} \leftarrow b_{tentative\_cand}$ 
59:   return  $b_{cand}$ 
60: end

```

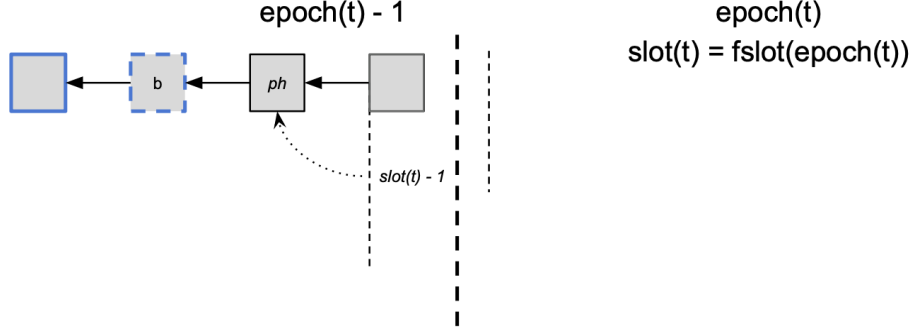


Figure 3: Configuration at the start of a new epoch ($\text{slot}(t) = \text{fslot}(\text{epoch}(t))$).

Case 2: Not the beginning of the epoch. If $\text{slot}(\text{now}^v) > \text{fslot}(\text{epoch}(\text{now}^v))$ (Figure 4), then the rule instead checks:

$$\begin{aligned} & \text{vs}(\text{phead}, \text{now}^v) \geq \text{epoch}(\text{now}^v) - 2, \\ & \text{willNoConflictingChkpBeJustified}^v(\text{C}(\text{head}, \text{epoch}(\text{now}^v))), \\ & (\text{GU}(\text{phead}) \geq \text{epoch}(\text{now}^v) - 1 \text{ or } \text{GU}(\text{head}) \geq \text{epoch}(\text{now}^v) - 1). \end{aligned}$$

In this case, in addition to the check executed for the case above, we also check that no checkpoint from the current epoch and conflicting with the current canonical chain can ever be justified and that the epoch of the greatest unrealized justified checkpoint of either the current or previous head is no older than the previous epoch. These checks ensure that whatever block we confirm, it cannot be filtered out in either the current or next epoch. This is because, as we cross into the next epoch $\text{epoch}(\text{now}^v + 1)$, the epoch of the voting source of either the previous or the current head will be the previous epoch $\text{epoch}(\text{now}^v - 1)$. Then, it is possible to prove by induction that a block confirmed under these conditions will never be filtered out.

The two extra conditions checked in the second case are not required in the first case because, if a block b is canonical for the entirety of the current epoch, then we know that no checkpoint from the current epoch conflicting with it can ever be justified and, by Assumption 3, we know that if enough attestations to justify its checkpoint from the previous epoch have been sent, then they will be included in a chain extending b before the current epoch ends, which means, that by the end of the current epoch we will have a block b' , descendant of b , with the epoch of its greatest unrealized checkpoint being from the previous epoch.

Previous Epoch - Second if Again, we consider two cases depending on whether the current time is the beginning of *first slot* of the current epoch.

Case 1: Beginning of the epoch. In this case, we proceed down the canonical chain as long as the condition $\text{isOneConfirmed}(b', \cdot)$ holds for every block b' that we encounter.

Say that $b_{\text{tentative_cand}}$ is the last such block and assume for now that the epoch of b_{cand} is the previous epoch.

In this case, we consider $b_{\text{tentative_cand}}$ confirmed as long as

$$\text{vs}(b_{\text{tentative_cand}}, \text{now}^v) \geq \text{epoch}(\text{now}^v) - 2$$

Otherwise, we do not update the latest confirmed block.

Note that if $\text{isOneConfirmed}(b_{\text{tentative_cand}}, \cdot)$ holds, it means that at least one honest validator voted for $b_{\text{tentative_cand}}$, which implies that such an honest validator has also gossiped $b_{\text{tentative_cand}}$ by the time they sent their vote. Given our network assumptions, this ensures that $b_{\text{tentative_cand}}$ is in the view of any

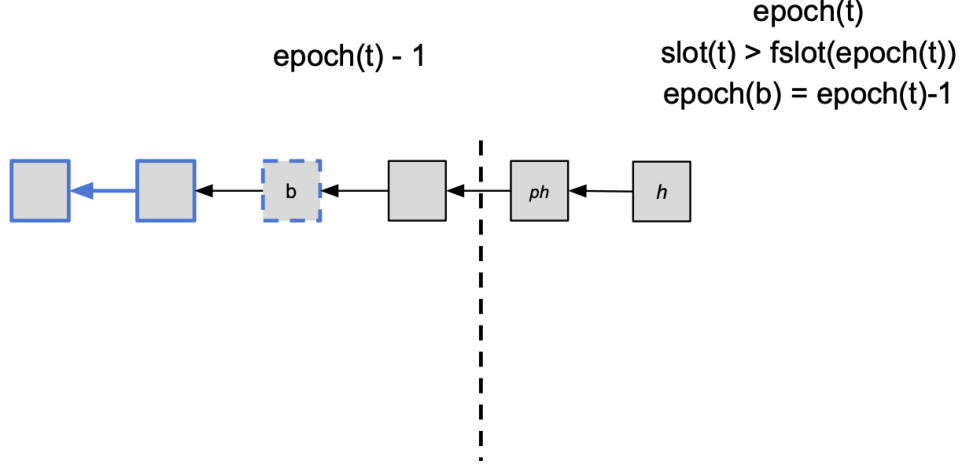


Figure 4: Here, $\text{slot}(t) > \text{fslot}(\text{epoch}(t))$ and $\text{epoch}(b) = \text{epoch}(t) - 1$, meaning we are not at the first slot of the epoch, but the confirmation process continues for a block belonging to the previous epoch.

honest validator at the current time. Then, the condition on the voting source ensures that $b_{\text{tentative_cand}}$ will not be filtered out in the current epoch. By an inductive argument, it is then possible to prove that it will never be filtered out.

Not at the beginning of the epoch. If we were not in the first slot of the current epoch, then we would also require that

$$\begin{aligned} \text{GU}(\text{head}) &\geq \text{epoch}(\text{now}^v) - 1, \\ \text{willNoConflictingChkpBeJustified}^v(C(\text{head}, \text{epoch}(\text{now}^v))) \end{aligned}$$

Note that these are the same conditions we check for in the first if when we are not at the beginning of the current epoch.

Current Epoch - Second if First, note that we can only receive a valid block from the current epoch if we are not the beginning of the current epoch. Line 47 then implies that we check that

$$\text{GU}(\text{head}) \geq \text{epoch}(\text{now}^v) - 1$$

Let us start with the case that the latest confirmed block is from the previous epoch, but, as noted above, we are not at the beginning of the current epoch. Note that due to Line 51, as we proceed down the canonical chain, the first time that we encounter a block b_i that belongs to the the current epoch, we confirm it and keep moving only if we can determine that its checkpoint will be eventually justified via $\text{willChkpBeJustified}_v(C(b_{\text{temp}}))$ (Figure 5).

This and Assumption 3 ensure that, by the beginning of the epoch after the next, such a checkpoint will be justified in a chain extending b_i . This ensures that b_i will not be filtered out from there on. We also already know that b_i will not be filtered out either the current or next epoch because of the condition $\text{GU}(\text{head}) \geq \text{epoch}(\text{now}^v) - 1$ that we have already checked.

Note from Line 57 that when the b_{temp} is from the current epoch (Figure 6), no further check is performed to confirm it at the end of the while loop of Line 49.

Inactive state. Before enabling the active state, the function first checks whether the current confirmation candidate b_{cand} is still consistent with the canonical view. When any of the following holds:

$$\text{epoch}(b_{\text{cand}}) < \text{epoch}(\text{now}^v) - 1 \quad \text{or} \quad b_{\text{cand}} \not\leq \text{head}$$

$$\text{or} \quad \left(\text{slot}(\text{now}^v) = \text{fslot}(\text{epoch}(\text{now}^v)) \text{ and } \neg(is_chain_one_confirmed^v(b_c, \text{GU}^{\text{st}(\text{slot}(t)-1), v})) \text{ and } b_c \geq \text{GU}^{\text{st}(\text{slot}(t)-1), v} \right),$$

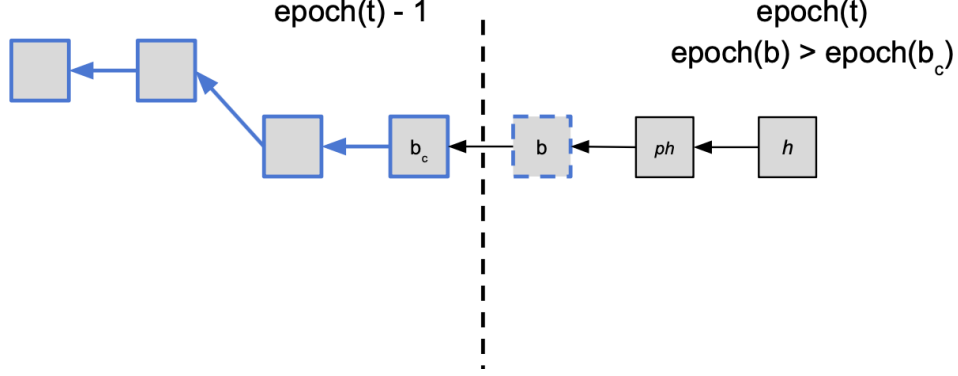


Figure 5: Cross-epoch confirmation attempt. Here, $\text{epoch}(b) > \text{epoch}(b_c)$, meaning the candidate block b lies in a later epoch than the last confirmed block b_c . The algorithm checks whether this block's checkpoint can be justified soon; if not, confirmation stops before entering that region.

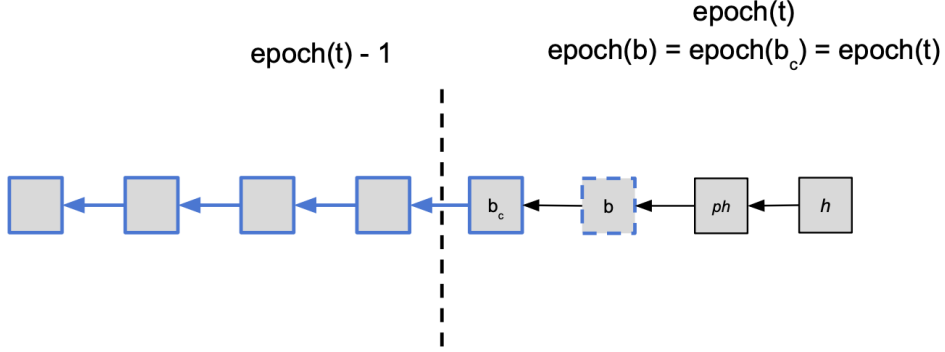


Figure 6: Here, $\text{epoch}(b) = \text{epoch}(b_c) = \text{epoch}(t)$, meaning both the candidate and last confirmed blocks belong to the current epoch.

the function determines that the current confirmation candidate is outdated or inconsistent. For instance, *is_chain_one_confirmed* verifies at the beginning of every epoch that every previously confirmed block still satisfies the confirmation predicate (we call these “reconfirmations”)⁶.

Specifically, the check succeeds if *any* of the following holds: the latest confirmed block is too old ($\text{epoch}(b_{\text{cand}}) < \text{epoch}(\text{now}^v) - 1$); or it is no longer on the canonical chain ($b_{\text{cand}} \not\prec \text{head}$); or, at the first slot of the epoch, the rule cannot reconfirm all previously confirmed blocks, namely

$$\text{slot}(\text{now}^v) = \text{fslot}(\text{epoch}(\text{now}^v)) \wedge \neg \left(\text{is_chain_one_confirmed}^v(b_c, \text{GU}^{\text{st}(\text{slot}(t)-1),v}) \right) \wedge b_c \succeq \text{GU}^{\text{st}(\text{slot}(t)-1),v}.$$

If this ever fails, or because any of the surrounding consistency does not hold, the rule immediately resets the confirmed block to the latest finalized one:

$$b_{\text{cand}} \leftarrow \text{block}(\text{GF}_{\text{now}}^v).$$

Intuitively, this recovery step acts as a *synchronization fallback*: it discards any potentially stale confirmation and anchors the process back to a finalized checkpoint, ensuring that future confirmations resume from a globally consistent and irreversible state, i.e., the greatest finalized checkpoint.

⁶Reconfirmation must use the same balances as the initial confirmation. Moreover, as stated in the “Per-block check” paragraph, the checkpoint used for the first *isOneConfirmed* evaluation must be an ancestor of the checkpoint any honest validator will later use in fork-choice. Hence, the first evaluation uses the greatest unrealized justified checkpoint seen at the beginning of the last slot of the previous epoch, while reconfirmation uses the one seen at the beginning of the last slot two epochs ago.